MULTI-THREADED WEB SERVER

Presented by:
Dev Patel-241IT023
Devansh Shah-241IT024
Vivaan Sharma-241IT081

INTRODUCTION

Multithreaded Web Server with Stateless API Gateway

- Modern web apps require efficient handling of concurrent requests.
- Traditional servers face bottlenecks and high resource overhead.
- Project: A custom C++ Multithreaded Web Server with a Stateless API Gateway.
- Uses thread-pool concurrency to manage simultaneous requests.
- Built to maximize scalability, efficiency, and performance.

MOTIVATION

Need for Custom Infrastructure

- Internet growth demands high-performance infrastructure.
- Older servers struggle with massive simultaneous connections.
- "Thread-per-request" models cause high memory and CPU overhead.
- Stateful systems make horizontal scaling difficult.
- Goal: A lightweight, scalable system that maximizes hardware utilization.

LITERATURE

Research Papers:

1.The Classic Concurrency Paper

Title: "SEDA: An Architecture for Well-Conditioned, Scalable Internet Services"Authors: Matt Welsh, David Culler, and Eric BrewerPublication: ACM Symposium on Operating Systems Principles (SOSP), 2001.

- "This paper validates my core architectural decision to reject the traditional 'thread-per-request' model, which I avoided to prevent memory exhaustion and CPU context-switching crashes under heavy load."

2. The Foundation of Web Server Performance

Title: "Flash: An efficient and portable Web server"Authors: Vivek S. Pai, Peter Druschel, and Willy ZwaenepoelPublication: USENIX Annual Technical Conference, 1999.

- Provides the foundational benchmarking evidence that backs my use of a fixed-size Thread Pool to achieve maximum hardware utilization compared to event-driven single threads.

LITERATURE

Research Papers:

3. Modern Thread Pool Optimization

Title: "Performance Evaluation of Thread Pool Configurations in the Run-time Systems of Integration Platforms"Authors: D. L. Freire, et al.Publication: International Journal of Business Process Integration and Management, 2021.

- Empirically proves my approach of using std::thread::hardware_concurrency() to dynamically size the thread pool to the exact number of CPU cores for optimal execution time and zero CPU thrashing.

4. API Gateway & Stateless Design

Title: "Microservices Architecture: Design Patterns, Scalability, and Inter-Service Communication Strategies"Authors: Niharika SinghPublication: International Journal of Computer Sciences and Engineering, 2024.

- Confirms that my implementation of a Stateless API Gateway for front-line rate-limiting is the modern industry standard for achieving fault isolation and horizontal scalability

LITERATURE

Review of Server Architectures

- Thread-per-request: Creates a new OS thread per user. High overhead, poor scaling.
- Event-driven (Node.js): Non-blocking I/O. Efficient, but complex to implement.
- Stateful architectures: Stores session data on servers. Hard to scale horizontally.
- Modern stateless architectures: Independent requests. Superior scalability and reliability.

OUTCOME OF LITERATURE

Architectural Decisions

- Existing models struggle with thread overhead or scaling limits.
- Highly efficient systems require:
  - Controlled concurrency.
  - Efficient resource management.
  - Stateless processing.
- Result: Designed a multithreaded server combining a task queue with a stateless API gateway.

PROBLEM STATEMENT

The Core Problem

- Traditional servers fail under thousands of concurrent requests.
- Excessive thread creation causes CPU and memory exhaustion.
- Stateful sessions restrict seamless horizontal scaling.
- Problem: Design an architecture that handles high concurrency with optimal resource use, scalability, and low latency.

PROJECT OBJECTIVES

- Build a Multithreaded Web Server using raw POSIX sockets.
- Implement Thread Pool concurrency to eliminate thread-creation overhead.
- Develop a Stateless API Gateway for scalable, secure routing.
- Improve system scalability and overall request throughput.
- Create a real-time dashboard to visualize server activity.

RESULT AND ANALYSIS

1. Performance & Load Testing
2. Successfully handled thousands of concurrent client connections.
3. Thread pool drastically reduced CPU context-switching overhead.
4. Stateless design enabled rapid, independent request processing.
5. Dashboard provided real-time visibility without slowing the backend.
6. Demonstrated high throughput (~7,600 Req/Sec) with minimal latency.

PROGRESS

C++ Server Diagnostics
Uptime: 63s
Live • Port 8080
REQ / SEC
12
ACTIVE CONNS
1
AVG LATENCY
0ms
200 OK
632
ERRORS / BLOCKED
208,684
LIVE TRAFFIC OVERVIEW
Total
Success
Errors
THREAD POOL
Queue: 0
Worker 0 IDLE
Worker 1 IDLE
Worker 2 IDLE
Worker 3 IDLE
Worker 4 IDLE
Worker 5 IDLE
Worker 6 IDLE
Worker 7 IDLE
Worker 8 IDLE
Worker 9 /metrics
CLIENT IP ADDRESS
TOTAL REQUESTS
LAST REQUESTED PATH
STATUS
127.0.0.1
209,316
/metrics
OK
Ping /health
Fetch Users
Slow Request (800ms)
🔥 Trigger Burst (10x)

PROGRESS

```
Transfer/sec:      1.66MB
devansh@Devanshs-MacBook-Air a % make load-test
Running 30s load test: 12 threads, 400 connections
wrk -t12 -c400 -d30s http://localhost:8080/health
Running 30s test @ http://localhost:8080/health
  12 threads and 400 connections
  Thread Stats   Avg      Stdev     Max   +/- Stdev
    Latency     1.96ms    3.86ms  99.00ms   99.50%
    Req/Sec     3.08k     1.64k    8.38k    67.33%
  218293 requests in 30.09s, 52.28MB read
  Socket errors: connect 157, read 0, write 0, timeout 0
  Non-2xx or 3xx responses: 218093
Requests/sec:   7255.30
Transfer/sec:      1.74MB
devansh@Devanshs-MacBook-Air a % make load-test
```

CONCLUSION

- Conclusion
- Demonstrates a scalable, custom-built web server architecture.
- Successfully combines thread pool concurrency with stateless communication.
- Tangibly improves hardware performance and memory management.
- Monitoring dashboard greatly enhances system observability.
- Proves modern concurrency techniques drastically improve server performance.

REFERENCES

- Andrew S. Tanenbaum – Computer Networks
- W. Richard Stevens – Unix Network Programming
- HTTP/1.1 Protocol Specification (RFC 2616)
- POSIX Threads Documentation
- Research papers on Multithreaded Web Servers and Scalable Architectures

THANK YOU